

ผมสามารถเอา model ใน best_model.zip ของ fold 5 ไป fintune ต่อกับสินทรัพย์อื่นๆในอดีตได้ไหม ? เช่น หุ้นที่อยู่มายาวๆในอดีตอย่าง KO, IBM, PG, JPM, XOM ที่เคยผ่านยุค Nifty-Fifty และ Stagflation มา จากนั้นก็เอาไป Fintuning ต่อกับหุ้นยุคใหม่ๆ เช่น GOOG, RCLB, SNDK, DELL ฯลฯ อะไรพวกนี้นะ ผมอยากรู้ว่าสามารถทำให้ Agent ส่งต่อประสบการณ์การเทรดเหมือนกับมนุษย์ที่มีประสบการณ์เทรดหุ้นหลายตัวมานับไม่ถ้วน และอยู่ในตลาดมานานหลายสิบปีได้ไหม

ในทางเทคนิคด้วยสถาปัตยกรรมของ PPO_V3 ปัจจุบัน คุณไม่สามารถนำ best_model.zip ไป Fine-tune แบบนั้นตรง ๆ ได้ทันที และการนำโมเดลไปเทรนเรียงตามลำดับเวลา (เก่าไปใหม่) จะไม่ทำให้ Agent กลายเป็น "เขียนหุ้นผู้มากประสบการณ์" แต่จะเกิดอาการ Catastrophic Forgetting (การลืมความรู้เดิมอย่างสิ้นเชิง)

3 อุปสรรคเชิงเทคนิคที่ทำให้ PPO_V3 นำไป Fine-tune ข้ามยุคตรง ๆ ไม่ได้

- **Architectural Lock & Semantic Misalignment:** โมเดล MLP ใน V3 ถูกออกแบบให้ Observation และ Action Space ผูกติดกับมิติเฉพาะของ 5 สินทรัพย์หลัก (SPY, QQQ, DIA, TLT, GLD) หากคุณป้อนข้อมูล 5 หุ้นตัวใหม่ (เช่น IBM, KO, PG, JPM, XOM) โครงข่ายประสาทเทียมจะตีความว่า JPM คือ TLT (พันธบัตร) และ XOM คือ GLD (ทองคำ) ตามตำแหน่ง Index เดิม ทำให้การตัดสินใจด้านความเสี่ยงและน้ำหนักเพี้ยนไปทั้งหมด
- **Catastrophic Forgetting (ลืมวิกฤตอดีตจนหมดสิ้น):** โครงข่ายประสาทเทียมที่ใช้ Policy Gradient จะปรับค่าน้ำหนัก (Weights) ตามชุดข้อมูลล่าสุดเสมอ หากคุณเทรนยุค Nifty-Fifty / Stagflation (ทศวรรษ 1970) จนเชี่ยวชาญ แล้วนำโมเดลมา Fine-tune ต่อด้วยหุ้นยุคใหม่ (GOOG, RCLB) Gradient ชุดใหม่จะเข้าไปเขียนทับ (Overwrite) พารามิเตอร์เดิม ทำให้ Agent ลืมวิธีเอาตัวรอดในสภาวะเงินเฟ้อสูงจนหมดสิ้น
- **Data & Regime Incompatibility:** ตลาดหุ้นยุค 1970 ไม่มีดัชนี VIX (VIX เริ่มคำนวณจริงปี 1993), โครงสร้างสภาพคล่องเป็นแบบ Floor Trading ที่มี Bid-Ask Spread กว้างและค่าคอมมิชชันสูงลิ่ว ขณะที่ตลาดยุคปัจจุบันขับเคลื่อนด้วย HFT, Passive Indexing และ Options OTE การนำสภาวะสองยุคนี้มาต่อกันเป็นเส้นตรงจะทำให้โมเดลเรียนรู้สัญญาณรบกวน (Noise) มากกว่า Pattern ที่ใช้ได้จริง

แนวทางที่ Quant Hedge Fund ใช้สร้าง "Agent ระดับเก๋าเกม" (The Institutional Way)

หากต้องการสร้าง Agent ที่ซับซ้อนประสิทธิภาพของหุ่นนับร้อยตัวข้ามหลายสิบปี สถาบันการเงินจะไม่ใช้วิธี Sequential Fine-tuning แต่จะเปลี่ยนไปใช้แนวคิด **Pre-trained Foundation Model for Trading**

- **Asset-Agnostic / Shared-Weight Architecture:**
 - เลิกใช้โมเดลที่รับ Flat Vector ของพอร์ตทั้งก้อน
 - เปลี่ยนเป็นโครงสร้างแบบ **Per-Asset Encoder** ตัวเดียวที่ใช้ Weight ร่วมกันกับหุ้นทุกตัว (Input ไม่ใช่ชื่อหุ้น แต่เป็นฟีเจอร์สัมพันธ์เชิงสถิติ เช่น Z-Score Return, Relative RSI, Volatility Percentile, และ Normalized Momentum)
 - โมเดลจะมองหุ้นทุกตัวเป็นเพียง "ชุดพฤติกรรมตัวเลข" ทำให้สามารถเทรนหุ้นพร้อมกัน 500 ตัวในอดีต และนำโมเดลเดียวกันมาประเมินหุ้น IPO ตัวใหม่ได้ทันทีโดยไม่ต้องเปลี่ยนโครงสร้าง Action Space
- **Joint Pre-Training (เทรนพร้อมกันทุกยุค แทนการเทรนทีละสเต็ป):**
 - ดึงข้อมูลหุ้นกว่า 500–1,000 ตัวย้อนหลัง 50 ปี (รวมทั้งหุ้นที่ล้มละลายหรือถูกเพิกถอน เพื่อป้องกัน Survivorship Bias)
 - ในแต่ละ Rollout Batch ให้สุ่ม Sampling ข้ามมิติเวลาและกลุ่มอุตสาหกรรม (เช่น ผสมข้อมูลปี 1974, 2000, 2008, 2020 เข้าด้วยกัน) เพื่อบีบให้ Policy Network สกัดเฉพาะ Latent Features ที่เป็นสากลและทนทานต่อทุกสภาวะตลาด
- **Domain Randomization & Continual Learning:**
 - ใช้เทคนิค Regularization เช่น **Elastic Weight Consolidation (EWC)** เพื่อล็อกค่าน้ำหนักที่มีความสำคัญต่อการเอาตัวรอดในวิกฤตอดีตไม่ให้ถูกแก้ไขตอนปรับจูนเข้ากับข้อมูลยุคใหม่

ข้อเสนอแนะในการวางกรอบงาน (Architecture Separation)

อย่าพยายามรวมโจทย์ "จัดพอร์ต Macro สินทรัพย์หลัก" เข้ากับ "การคัดเลือกหุ้นรายตัว (Stock Picking)" ไว้ในโมเดลเดียวกัน:

- **PPO_V3 (Macro Tactical Allocator):** คงไว้สำหรับจัดสรร Asset Allocation ข้ามสินทรัพย์หลัก (SPY, QQQ, DIA, TLT, GLD) เพื่อคุม Beta และความผันผวนรวมของพอร์ต
- **Research Track ตัวใหม่ (Cross-Sectional Alpha Selector):** หากต้องการลุยหุ้นเดี่ยว (ทั้งกลุ่ม Value ยุคเก่าและ Growth ยุคใหม่) ให้สร้างโปรเจกต์แยกเป็นโมเดล Ranking / Cross-Sectional Momentum โดยใช้ Shared-Weight Network ซึ่งจะตรงกับสมมติฐานของการส่งต่อประสิทธิภาพการเทรดหุ้นข้ามยุคสมัยได้อย่างถูกต้องตามหลักสถิติ

ช่วยอธิบายเรื่อง Pre-trained Foundation Model for Trading ให้ลึกกว่านี้หน่อยได้ไหม

Pre-trained Foundation Model for Trading คือการเปลี่ยนกระบวนทัศน์จากการสร้างโมเดลที่ "จดจำค่าเฉพาะของหุ้นไม่กี่ตัว" ไปสู่การสร้างโมเดลแกนกลางที่เข้าใจ "ไวยากรณ์และพฤติกรรมสากลของตลาดการเงิน" (Financial Market Dynamics) โดยเรียนรู้จากสินทรัพย์นับพันตัวข้ามประวัติศาสตร์หลายสิบปีในลักษณะเดียวกับที่ Large Language Model (LLM) เรียนรู้โครงสร้างภาษา ก่อนนำไปปรับแต่ง (Fine-tune) เข้ากับกลยุทธ์เฉพาะตัว

1. สถาปัตยกรรม Asset-Agnostic (ตัดชื่อ Ticker ทั้งทั้งหมด)

หัวใจของ Foundation Model สำหรับการเงินคือการไม่ผูกมัดกับ Ticker ใดเลย โมเดลจะไม่ว่าสินทรัพย์ที่กำลังอ่านคือ JPM, GOOG หรือราคาทองคำปี 1970:

- **Patching Representation:** ตัดแปลง Time Series ให้กลายเป็น "Tokens" เช่นเดียวกับโมเดลตระกูล PatchTST หรือ iTransformer โดยตัด Time Series ความยาว \$L\$ วัน ย่อยออกเป็น Patches เล็ก ๆ (เช่น ท่อนละ 5 หรือ 10 วัน) เพื่อให้ Attention Mechanism สามารถจับความสัมพันธ์ของแนวโน้มระยะสั้นและวงจระยะยาวได้พร้อมกัน
- **Cross-Sectional Standardization:** ฟีเจอร์ทั้งหมดจะถูกแปลงเป็นรูปสถิติสัมพัทธ์ (Relative Stat) เสมอ เช่น Percentile Rank เทียบกับตลาดในวันนั้น หรือ

Volatility-Normalized Return ส่งผลให้พฤติกรรมหุ้นพลังงานยุค Stagflation 1973 มีสเกลตัวเลขที่โครงข่ายประสาทเทียมเทียบเคียงกับหุ้นเทคโนโลยียุค 2020 ได้โดยตรง

2. กลไกการ Pre-train (Self-Supervised Learning)

โมเดลไม่ได้ถูกสอนด้วย PPO หรือผลกำไร/ขาดทุนตั้งแต่แรก แต่เรียนรู้ความเข้าใจในโครงสร้างข้อมูลตลาดผ่านงาน Self-Supervised Learning บนชุดข้อมูลขนาดมหาศาล:

- **Masked Time-Series Modeling:** สุ่มปิดบัง (Mask) ข้อมูลราคา ปริมาณการซื้อขาย หรืออินดิเคเตอร์บางช่วงเวลา (เช่น ซ่อน 30% ของข้อมูล 90 วันย้อนหลัง) แล้วบังคับให้โมเดลทำนายข้อมูลที่หายไปคืนมา บังคับให้โครงข่ายต้องเข้าใจ Market Inefficiency, Mean Reversion และ Regime Transitions
- **Contrastive Learning (InfoNCE):** ดึง Representation ของสภาวะตลาดที่มีโครงสร้างคล้ายกัน (เช่น สภาพคล่องหดตัว, อัตราผลตอบแทนพันธบัตรผกผัน) ให้เข้ามาอยู่ใกล้กันใน Latent Space และผลักสภาวะที่ต่างกัน (เช่น สภาพคล่องล้นตลาด) ให้ห่างจากกัน
- **Distributional Forecasting:** แทนที่จะทำนายราคาเดียว (Point Estimate) โมเดลจะถูกสอนให้ทำนายการกระจายตัวของผลตอบแทนในอนาคต (Return Density) ซึ่งครอบคลุมทั้ง Volatility, Skewness และ Fat Tails

3. การนำไป Fine-tune ด้วย RL โดยไม่เกิด Catastrophic Forgetting

เมื่อได้ Foundation Backbone ที่ผ่านการ Pre-train กับข้อมูลหุ้นหลายพันตัวตลอด 50 ปี สถาบันการเงินจะใช้เทคนิค **Parameter-Efficient Fine-Tuning (PEFT)** เพื่อนำมาใช้เทรดจริง:

มิติการเปรียบเทียบ	Traditional End-to-End RL (แบบ PPO_V2/V3)	Foundation Model + RL Policy Head
Input Data	เจาะจง 3–5 สินทรัพย์ (Fixed Dimension)	ข้อมูล Panel ของสินทรัพย์ก็ได้ (Flexible)
Data Efficiency	ต้องการ Rollout นับล้าน Steps เพื่อเรียนรู้ Pattern พื้นฐาน	มี Zero-Shot/Few-Shot Capabilities เรียนรู้ Alpha ใหม่ได้รวดเร็ว
ความทนทานข้าม Regime	มัก Overfit กับยุคที่ใช้เทรน (ลืมนักลงทุนในอดีต)	ซึมซับวิกฤตทุกรูปแบบไว้ใน Latent Space เรียบร้อยแล้ว
การปรับจูน (Fine-tuning)	เทรนทับทั้งโมเดล เสี่ยงเกิด Catastrophic Forgetting	แช่แข็ง (Freeze) โครงข่ายแกนกลาง 95% และจูนเฉพาะ Adapter/LoRA Head

กระบวนการทำงานในขั้นตอน Production

เมื่อนำไปปรับใช้ในระบบเทรด โครงข่ายส่วน Backbone จะทำหน้าที่เป็น **Feature & Regime Extractor** ขั้นสูง คอยส่ง Latent State Vectors ออกมา ณ สิ้นวัน จากนั้น Actor-Critic Head ขนาดเล็กที่ผ่านการ Train ด้วย PPO จะนำ Vector เหล่านี้ไปแปลงเป็นสัดส่วนน้ำหนักพอร์ตโฟลิโอ (Target Weights)

แนวทางนี้ช่วยรักษา "ภูมิคุ้มกันตลาด" ที่สั่งสมมาจากวิกฤตหลายทศวรรษในอดีตไว้ได้อย่างสมบูรณ์ โดยที่สัญญาณการซื้อขายในปัจจุบันจะไม่เข้าไปทำลายความทรงจำระยะยาวของระบบเหมือนการบังคับเทรนต่อแบบเดิม

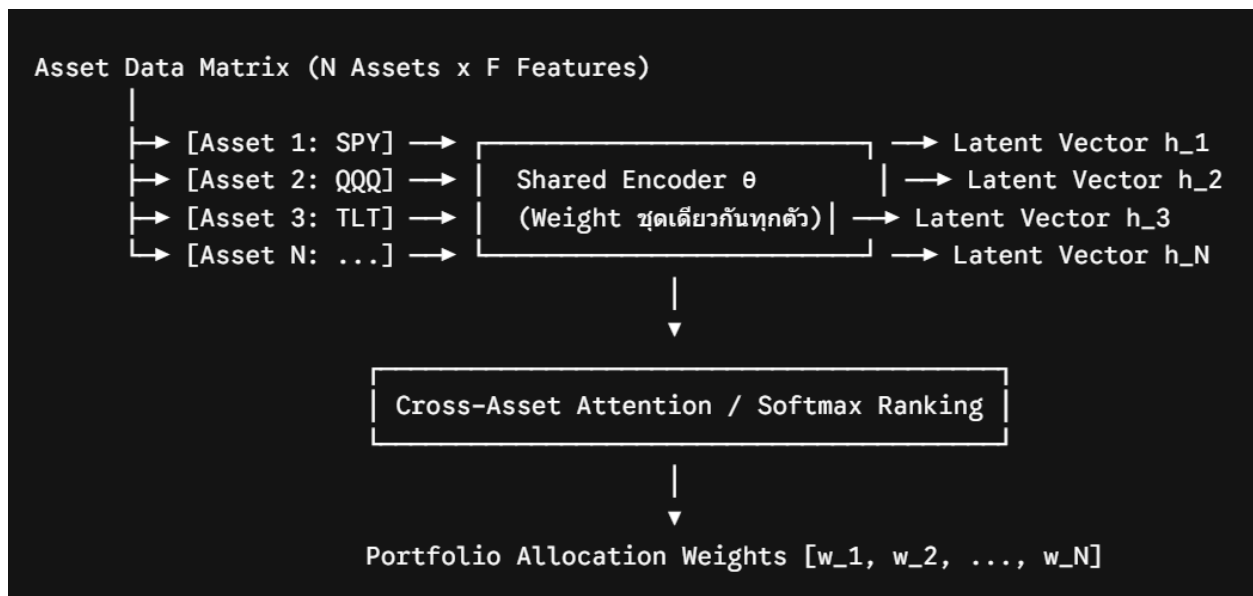
อธิบายเรื่อง Asset-Agnostic / Shared-Weight Architecture, Joint Pre-Training (เทรนพร้อมกันทุกยุค แทนการเทรนทีละสแต็ป), Domain Randomization & Continual Learning ให้ละเอียดกว่านี้หน่อย

1. Asset-Agnostic / Shared-Weight Architecture

โมเดล RL หรือ Deep Learning ทั่วไปสำหรับพอร์ตโฟลิโอ (เช่น PPO_V2/V3) มักป้อนข้อมูลแบบ Concatenate ทุกสินทรัพย์เป็น Flat Vector แฉวยาว เช่น

$[feat_{SPY}, feat_{QQQ}, \dots]$

วิธีนี้ทำให้โครงข่ายผูกมัดติดกับ Ticker นั้นโดยตรง (Weight ของ Node ที่ 1 ทำหน้าที่อ่านเฉพาะ SPY เสมอ) สถาปัตยกรรมแบบ **Shared-Weight** แก้ปัญหานี้ด้วยการแปลงระบบให้เป็น **Permutation-Equivariant System**



Weight-Sharing Representation: ไม่ว่าจะเป็นหุ้นตัวใด ข้อมูลของสินทรัพย์เดี่ยวจะถูกส่งผ่าน

Encoder Φ_θ

ซัดเดียวกันทั้งหมด:

$$h_{i,t} = \Phi_{\theta}(x_{i,t})$$

โดย Θ คือพารามิเตอร์ชุดเดียวกันสำหรับทุกสินทรัพย์ ทำให้โมเดลมองสินทรัพย์เป็นเพียง "ชุดพฤติกรรมตัวเลข" (Time-Series Pattern) ไม่ใช่ชื่อย่อหุ้น

- **Cross-Sectional Standardization:** อินพุตจะไม่ใช้ราคาหรือค่าสัมบูรณ์ตรง ๆ แต่ใช้สถิติสัมพันธ์ ณ เวลา t เช่น:
 - *Cross-Sectional Percentile:* แปลง Momentum หรือ RSI ให้เป็นค่า Rank ข้ามสินทรัพย์ทั้งหมดตั้งแต่ 0.0 ถึง 1.0
 - *Volatility-Normalized Relative Return:* ส่วนต่างผลตอบแทนเทียบกับค่าเฉลี่ยของตลาด หาดด้วย Realized Volatility ของตัวมันเอง
- **Permutation Equivariance ผ่าน Attention Mechanism:** เวกเตอร์

$h_{i,t}$

ของทุกตัวจะถูกส่งเข้า Self-Attention Layer เพื่อคำนวณความสัมพันธ์ระหว่างกัน หากคุณสลับตำแหน่งระหว่าง Asset A และ Asset B ใน Input ลำดับของ Output Weights ก็จะสลับตามโดยที่ค่าการจัดสรรไม่เพี้ยน ระบบจึงรองรับสินทรัพย์ตั้งแต่ 5 ตัว ไปจนถึง 500 ตัว และรองรับหุ้น IPO หรือถอดหุ้นที่ Delist ออกได้ทันทีโดยไม่ต้องแก้ไขโมเดล

2. Joint Pre-Training (เทรนพร้อมกันทุกยุค แทนการเทรนทีละสแต็ป)

การเทรนโมเดลไล่ตามลำดับเวลา เช่น เทรนยุค 1970 -> 1980 -> 2000 -> ปัจจุบัน จะล้มเหลวเสมอจาก **Catastrophic Forgetting** เพราะ Gradient ในยุคหลังจะเขียนทับ Parameter พื้นที่ที่เคยจดจำวิกฤตในอดีต การทำ **Joint Pre-Training** บังคับให้ Optimization ค้นหา **Invariant Latent Manifold** ที่ใช้ได้กับทุกยุคไปพร้อมกัน

มิติการทำงาน	Sequential Training (แบบเดิม)	Joint Pre-Training (แบบสถาบัน)
โครงสร้าง Batch	ข้อมูลเรียงตามเวลา t, t+1, t+2 ในยุคเดียว	ผสม Sampling ข้ามยุคใน 1 Update Step
เป้าหมาย Objective	ลด Loss เฉพาะสถานะตลาดปัจจุบัน	หา Minimum ร่วมของ Loss ข้ามทุก Regime
ความเสี่ยง	ปรับตัวเข้ากับ Regime ล่าสุด แต่ลืมหักัดอดีต	ต้องใช้ความจุเครือข่าย (Model Capacity) สูงขึ้น

•

Stratified Regime Batching: ในการเทรน 1 Rollout Update จะแบ่ง Workers ใน **SubprocVecEnv** ให้ดึงประวัติศาสตร์ตลาดจากคนละ Regime มาประมวลผลพร้อมกัน:

- *Worker 1–2:* สุ่มช่วง 1973–1974 (Stagflation: เงินเฟ้อสูง หุ่นและพันธบัตรร่วงพร้อมกัน)
- *Worker 3–4:* สุ่มช่วง 2000–2002 (Dot-Com Crash: Tech Valuation ย่อเย็บ แต่ Value/Bond ทำงาน)
- *Worker 5–6:* สุ่มช่วง 2008 (Global Financial Crisis: สภาพคล่องแห้งเหือด วิกฤตสถาบันการเงิน)
- *Worker 7–8:* สุ่มช่วง 2020–2021 (ZIRP & Quantitative Easing: สภาพคล่องล้น หุ่นเติบโตพุ่งแรง)

- **Mathematical Invariance:** การคำนวณ Gradient รวมจะเกิดจากการเฉลี่ย Loss ของสถานะที่ตรงข้ามกันอย่างสิ้นเชิง:

$$\nabla_{\theta} \mathcal{L}_{\text{joint}} = \frac{1}{K} \sum_{k=1}^K \mathbb{E}_{(s,a) \sim \mathcal{D}_k} [\nabla_{\theta} \mathcal{L}(s, a; \theta)]$$

โมเดลจะไม่สามารถ "โกง" ด้วยการจำเป็น Bias ของยุคใดยุคหนึ่งได้ ทางเดียวที่ Loss รวมจะลดลงคือ Policy ต้องสีกัด **Causal Structural Laws** ที่เป็นแก่นแท้ของระบบการ

เงินออกมาเท่านั้น (เช่น เมื่อ Real Yield พุ่งแรงพร้อม Credit Spread ขยายตัว สินทรัพย์เสี่ยงสูงทุกยุคจะพังทลาย)

3. Domain Randomization & Continual Learning

ตลาดการเงินไม่ใช่ระบบปิดที่คงที่ แต่มี **Non-Stationarity** สูงมาก ทั้งสองเทคนิคนี้ทำหน้าที่เป็นเข็มขัดนิรภัยและระบบภูมิคุ้มกันของโมเดลเมื่อต้องเผชิญหน้ากับสถานะที่ไม่เคยพบมาก่อน (Out-of-Distribution Data)

Domain Randomization (สร้างภูมิคุ้มกันสภาวะจำลอง)

หยิบยืมแนวคิดมาจาก Sim-to-Real Transfer ในงานวิทยาการหุ่นยนต์ โดยการป้อน Noise และความปั่นป่วนสังเคราะห์เข้าไปใน Environment ระหว่างการเทรน เพื่อป้องกันไม่ให้โมเดลจำลองตัวเลขละเอียดอ่อนจนเกินไป:

- *Slippage & Cost Perturbation*: สุ่มอัตรา Friction ในแต่ละ Episode ตั้งแต่ 5 bps ไปจนถึง 50 bps เพื่อบีบให้ Policy หลีกเลี่ยงกลยุทธ์ที่พึ่งพา Turnover สูงโดยไม่จำเป็น
- *Stochastic Volatility / Jump Injections*: สุ่มใส่ Synthetic Jump Diffusion (เช่น Flash Crash จำลอง -5% ถึง -10% ภายในวันเดียว) ลงในบางช่วงเวลา เพื่อสอนให้ Risk Overlay คุมความเสี่ยงอยู่ตลอดเวลา
- *Macro Reporting Lag Jitter*: เพิ่มสัญญาณรบกวนในตัวเลข Macro และสุ่มเลื่อนวันประกาศข้อมูล (Lag Randomization) ป้องกันไม่ให้ Agent อาศัยความแม่นยำของไทม์ไลน์ข้อมูลเศรษฐกิจมากเกินไป

Continual Learning (การเรียนรู้ตลอดชีพโดยไม่ลืมอดีต)

เมื่อนำระบบขึ้น Production แล้วต้องปรับตัวเข้ากับข้อมูลปัจจุบันอย่างต่อเนื่อง ระบบจะใช้เทคนิค **Elastic Weight Consolidation (EWC)** ร่วมกับ **Replay Buffer**:

- **Elastic Weight Consolidation (EWC)**: คำนวณ **Fisher Information Matrix (\$F\$)** เพื่อประเมินว่า Parameter ตัวไหนในโครงข่ายประสาทเทียมมีผลสำคัญอย่างยิ่งต่อการเอาตัวรอดในวิกฤตอดีต เมื่อมีการ Retrain หรือ Fine-tune ข้อมูลสดใหม่ จะมีการเพิ่ม Penalty ใน Loss Function:

$$\mathcal{L}_{\text{continual}}(\theta) = \mathcal{L}_{\text{current}}(\theta) + \sum_j \frac{\lambda}{2} F_j(\theta_j - \theta_j^*)^2$$

- Parameter ที่ควบคุมระบบลด Drawdown ในอดีต (Fj สูง) จะถูก "ล็อกหรือขยับได้ยากมาก" ส่วน Parameter ตัวอื่นที่เกี่ยวข้องกับ Noise สภาพคล่องทั่วไป (Fj ต่ำ) จะมีความยืดหยุ่นให้ปรับเข้ากับตลาดปัจจุบันได้อิสระ
- **Core Memory Replay Reservoir:** กันพื้นที่ 10–20% ของข้อมูลในแต่ละรอบ Retrain ไว้สำหรับป้องกัน "วิกฤตจำลองในประวัติศาสตร์" ซ้ำเข้าไปเสมอ เพื่อให้โมเดลรักษากฎมีคํ่ากันต่อเหตุการณ์ Black Swan ไว้อย่างถาวร

